EE5203 L03 Lab Guide

Process eight sensor values with small C functions - Basri Erdogan

Mission: Copy your completed L02 project, add four C files, clamp eight sensor values, calculate their average, and write `"LOW"`, `"OK"`, or `"HIGH"`. Prove the result with compiler output and debugger evidence.

Prepare before class

- ☐ Bring your working L02 Nucleo-F401RE project and USB data cable.
- ☐ Review function declarations, definitions, calls, array bounds, and `\0`.
- ☐ Download the L03 `starter/` folder from the course materials.
- ☐ Complete this entry ticket before the session warm-up:

```
uint16_t average_samples(uint16_t values[], int count);
```

1. Circle the return type, function name, and two parameters.
2. Explain why `count` is required.
3. State the valid indices when `count == 8`.

Learning objectives

By checkoff time, you can:

1. read the types, names, and parameters in a C declaration;
2. connect a declaration, definition, and call across C files;
3. loop through an array without using an invalid index;
4. write a C string that fits in its array and ends with `\0`;
5. use debugger evidence to justify a result and one controlled change.

Hardware and files

Item	Required value
Board	Nucleo-F401RE
Tools	STM32CubeMX + VS Code
Starting point	A working copy of the L02 project
Project name	`EE5203_L03_<studentnumber>`
Header files	`sensor_math.h`, `status_text.h`
Source files	`sensor_math.c`, `status_text.c`
Input	Eight fixed `uint16_t` samples; no ADC yet
Evidence	Build console, breakpoint, Watch/Expressions, Memory view

Keep all edits to generated `main.c` inside the appropriate `USER CODE` regions.

Checkpoint A - Add the four starter files

1. Copy the L02 project and rename the copy `EE5203_L03_<studentnumber>`.
2. Copy the starter headers into `Core/Inc` and sources into `Core/Src`; confirm each `.c` includes its own `.h`.
3. Add these includes inside `USER CODE BEGIN Includes` in `main.c`:

```
#include <stdbool.h>
#include <stdint.h>
#include "sensor_math.h"
#include "status_text.h"
```

4. Build the unmodified starter with zero errors, then identify where `clamp_u16` is declared, defined, and called.

Evidence for Checkpoint A: Show the four module files, a successful build, and the declaration/definition/call locations for one function.

Call the new functions from `main.c`

Inside `USER CODE BEGIN PV` in `main.c`:

```
#define SAMPLE_COUNT       8
#define STATUS_CAPACITY    8
#define SAMPLE_LIMIT       4095U
#define HIGH_THRESHOLD     2500U

uint16_t raw_samples[SAMPLE_COUNT] = {
    850U, 1250U, 4095U, 4200U,
```

```
    2050U, 90U, 3100U, 2800U
};

uint16_t bounded_samples[SAMPLE_COUNT];
char status_text[STATUS_CAPACITY];

uint16_t sample_average = 0U;
int samples_above = 0;
bool status_valid = false;
bool lab_done = false;
```

Inside `USER CODE BEGIN 2`, before the generated `while (1)`:

```
for (int i = 0; i < SAMPLE_COUNT; ++i) {
    bounded_samples[i] =
        clamp_u16(raw_samples[i], 0U, SAMPLE_LIMIT);
}

sample_average = average_samples(bounded_samples, SAMPLE_COUNT);
samples_above = count_above(bounded_samples,
                            SAMPLE_COUNT,
                            HIGH_THRESHOLD);
status_valid = make_status(status_text,
                           STATUS_CAPACITY,
                           sample_average);
lab_done = true;
```

Do not add a second infinite loop. Leave the generated `while (1)` in place.

Checkpoint B - Clamp and average the array safely

Complete these functions in `sensor_math.c`:

```
uint16_t clamp_u16(uint16_t value,
                   uint16_t low,
                   uint16_t high);

uint16_t average_samples(uint16_t values[],
                         int count);

int count_above(uint16_t values[],
                int count,
                uint16_t threshold);
```

Requirements:

- `clamp_u16` bounds every value into `0U…4095U`.
- Array loops use `i < count`; `average_samples` returns `0U` when `count == 0` and sums into a `uint32_t` so the total fits before division.
- `count_above` is strict: `values[i] > threshold` with `HIGH_THRESHOLD = 2500U`.

Before debugging, calculate the expected results:

Quantity	Prediction	Observed
`bounded_samples[3]`		
`sample_average`		
`samples_above`		

Evidence for Checkpoint B: Show a breakpoint inside `average_samples`, the changing index and accumulator, and the final three observed values.

Checkpoint C - Write `LOW`, `OK`, or `HIGH` and add `\0`

Complete this function in `status_text.c`:

```
bool make_status(char destination[],
                 int capacity,
                 uint16_t average);
```

Requirements:

- Thresholds: `"LOW"` below `1500U`, `"HIGH"` above `3000U`, otherwise `"OK"`; capacity is eight characters including the terminator.
- Check the required capacity before writing; write the `\0` explicitly.
- Return `true` only after a complete string is written; on insufficient capacity write an empty string when possible and return `false`.

Inspect `status_text` in both the Watch panel and the Memory view, and record the character and hexadecimal byte at indices 0 through 4.

Evidence for Checkpoint C: Show the readable string, the terminator byte, and `status_valid`.

Checkpoint D - Debugger evidence

Use a breakpoint at or immediately before `lab_done = true;`.

Predict the final values first, then inspect every result variable and confirm `lab_done` flips to `true` on resume. Collect evidence for each claim:

no index exceeded `7`; the `4200` sample became `4095`; the average used all eight clamped samples; the status text is null-terminated.

Checkpoint E - Individual modification

Each student receives one variant. State the prediction before editing, change only what is necessary, then collect fresh evidence.

Variant	Required change
A	Change the `90U` sample to `5000U`; predict the clamped value and new average.
B	Change `HIGH_THRESHOLD` from `2500U` to `2000U`; predict `samples_above`.
C	Change the `HIGH` status boundary from `3000U` to `2200U`; predict the string.
D	Call the array functions with `SAMPLE_COUNT - 1`; predict which sample is excluded and the new results.

Evidence for Checkpoint E: Demonstrate the new result and explain how the changed input or function call produced the observed output.

Acceptance checklist

- ☐ Build is clean; declarations and definitions match exactly.
- ☐ Array loops use explicit length and valid indices; the sum uses `uint32_t`.
- ☐ The status string fits, ends with `\0`, and reports success.
- ☐ Predictions, observations, and Checkpoint D evidence are recorded.
- ☐ The assigned individual modification is demonstrated.
- ☐ All `main.c` edits stay inside protected user-code regions.

Individual oral check

Be prepared for one question and one live change:

- What is the difference between declaring and defining a function?
- Why can `average_samples` not discover the caller's array length?
- Why `uint32_t` for the sum, and where is the `\0` byte in your status array?
- What does a successful build prove, and what does the debugger prove?
- If the linker reports `undefined reference`, which stage do you inspect?

Submit

Submit the four module files, the `main.c` user-code sections, one debugger screenshot of the final arrays and results, the completed tables, and a three-sentence explanation of your modification.

If you are stuck

Read the **first** compiler or linker message and compare the declaration with its definition; test one small input while watching `i`, `count`, and the accumulator (or the string's capacity and `\0` byte); record expected result, observed result, and the smallest next test.